

ESP 32 CODE:

```
1 #include <HardwareSerial.h>
2 #include <TinyGPSPlus.h>
3 #include <ESP32Servo.h>
4 #include <Wire.h>
5 #include <Adafruit MPU0805.h>
6 #include <Adafruit_Sensor.h>
7 #include <WiFi.h>
8 #include <HTTPClient.h>
9 #include <HTTPClientSecure.h>
10
11 #define I2C_ADDRESS 0x08
12
13 // ===== USER SETTINGS =====
14 const char* ssid = "Farish's A03";
15 const char* password = "987654321";
16
17 String botToken = "8577043634:AAFhyefn-Z3nZp_C1mEKBLv6t-nC0vc00";
18 String chatID = "722390435";
19
20 // ===== PIN MAP =====
21 #define GPS_RX 16
22 #define GPS_TX 17 // For MPU and LIDAR
23 #define SENSOR_SDA 14 // For MPU and LIDAR
24 #define SENSOR_SCL 15 // For MPU and LIDAR
25 #define SERVO_PIN 18
26 #define RELAY_PIN 23
27 #define LDR_PIN 33
28
29 // ===== OBJECTS =====
30 HardwareSerial gpsSerial(2);
31 TinyGPSPlus gps;
32 Servo mudgeServo;
33 Adafruit MPU0805 mpu;
34 TwoWire I2C_Sensors = TwoWire(1); // Dedicated bus for sensors
35
36 unsigned long fallCooldown = 0;
37 bool mpuAvailable = false;
38
39 void setup()
40 {
41   Serial.begin(115200);
42
43   // 1. Setup I2C Slave for Raspberry Pi (Default pins 21 & 22)
44   Wire.begin(I2C_ADDRESS);
45   Wire.onReceive(recvEvent);
46   Serial.println("I2C Slave (Pi 5) ready on pins 21/22");
47
48   // 2. Setup I2C Master for Sensors (Pins 14 & 15)
49   I2C_Sensors.begin(SENSOR_SDA, SENSOR_SCL, 100000);
50
51   // 3. Initialize Sensors on the Sensor Bus
52   if (!mpu.begin(0x68, I2C_Sensors))
53   {
54     Serial.println("MPU0805 Not Found!");
55     mpuAvailable = false;
56   }
57   else
58   {
59     Serial.println("MPU0805 Ready!");
60     mpuAvailable = true;
61   }
62
63   // 4. Other Hardware
64   gpsSerial.begin(9600, SERIAL_8N1, GPS_RX, GPS_TX);
65   pinMode(RELAY_PIN, OUTPUT);
66   pinMode(LDR_PIN, INPUT);
67   mudgeServo.attach(SERVO_PIN);
68   mudgeServo.write(0);
69
70   // 5. WiFi
71   WiFi.begin(ssid, password);
72   int timeout = 0;
73   while (WiFi.status() != WL_CONNECTED && timeout < 20)
74   {
75     delay(500);
76     Serial.print(".");
77     timeout++;
78   }
79   if (WiFi.status() == WL_CONNECTED)
80   {
81     sendTelegramMessage("System Online: Smart Cane Ready.");
82   }
83 }
84
85 void loop()
86 {
87   // GPS
88   while (gpsSerial.available() > 0)
89   {
90     gps.encode(gpsSerial.read());
91   }
92
93   // Light Control
94   int light = analogRead(LDR_PIN);
95   digitalWrite(RELAY_PIN, (light < 700) ? LOW : HIGH); //If LDR detect light below 700 threshold then, it will turn LOW. If not it will stay HIGH
96
97   // LIDAR (Pin 14 and 15)
```

[Download index package_esp32_index.json](#)

```

98 static unsigned long lastIdeaTime = 0;
99 if (millis() - lastIdeaTime > 100)
100 {
101     lastIdeaTime = millis();
102     int dist = readFlume();
103     if (dist > 30 && dist < 80) //distance between 30cm and 80cm
104     {
105         rudgeServo.write(90); //move 90 degrees
106         delay(200);
107         rudgeServo.write(0);
108     }
109 }
110
111 // MPU Fall Detection (Pin 14 and 15)
112 if (mpu.available())
113 {
114     sensors_event_t a, g, temp;
115     mpu.getEvent(&a, &g, &temp);
116     float totalAccel = sqrt(pow(a.acceleration.x, 2) + pow(a.acceleration.y, 2) + pow(a.acceleration.z, 2));
117
118     if ((isnan(totalAccel) && totalAccel > 30.0 && (millis() - fallCooldown > 5000)) //isnan refers to "Is the total acceleration a valid number" and the millis refers to the time more than 5 sec
119     {
120         sendTelegramAlert("FALL DETECTED");
121         fallCooldown = millis();
122     }
123 }
124 }
125
126 void receiveEvent(int howMany) //howMany tells the ESP32 device exactly how many bytes of data were sent by the Rasp pi 5
127 {
128     while (Wire.available())
129     {
130         int command = Wire.read(); // Read as Integer
131         Serial.print("I2C Command: ");
132         Serial.println(command);
133         if (command == 1) //receive 1 from rasp pi 5
134         {
135             sendTelegramAlert("SOS BUTTON PRESSED");
136         }
137     }
138 }
139
140 // ----- TELEGRAM & SENSORS -----
141
142 void sendTelegramAlert(String reason)
143 {
144     if (WiFi.status() == WL_CONNECTED)
145     {
146         String message = " 🚨 EMERGENCY 🚨 \nReason: " + reason;
147
148         if (gps.location.isInvalid())
149         {
150             message += "\n\nX Location:\nhttps://www.google.com/maps?q=" + String(gps.location.lat(), 6) + "," + String(gps.location.lng(), 6); // used to tell the location
151         }
152         else
153         {
154             message += "\n\nX No Fix Location;
155         }
156         sendTelegramMessage(message);
157     }
158 }
159
160 void sendTelegramMessage(String msg)
161 {
162     WiFiClientSecure client;
163     client.setInsecure();
164     HTTPClient https;
165     msg.replace(" ", "%20");
166     msg.replace("\n", "%0A");
167
168     String url = "https://api.telegram.org/bot" + botToken + "/sendMessage?chat_id=" + chatID1 + "&text=" + msg; //used to tell which telegram bot to used
169     if (https.begin(client, url))
170     {
171         int httpCode = https.GET();
172         https.end();
173     }
174 }
175
176 int readFlume() //basic example from arduino IDE to read TF1una
177 {
178     I2C_Sensors.beginTransmission(0x10);
179     I2C_Sensors.write(0x5A); I2C_Sensors.write(0x05); I2C_Sensors.write(0x00);
180     I2C_Sensors.write(0x01); I2C_Sensors.write(0x00);
181     I2C_Sensors.endTransmission();
182
183     I2C_Sensors.requestFrom(0x10, 9);
184     if (I2C_Sensors.available() >= 9)
185     {
186         I2C_Sensors.read(); I2C_Sensors.read();
187         int dl = I2C_Sensors.read();
188         int dh = I2C_Sensors.read();
189         while(I2C_Sensors.available()) I2C_Sensors.read();
190         return (dh * 256) + dl;
191     }
192     return -1;
193 }

```

RASPBERRY PI 5:

```
1 import os
2 import time
3 from smbus2 import SMBus
4 import subprocess
5 from gpiozero import Button
6 from pyhuskylens import Huskylens, ALGORITHM_OBJECT_RECOGNITION
7
8 # --- CONFIGURATION ---
9 UART_PORT = "/dev/serial0"
10 BAOO_RATE = 115200
11 I2C_BUS = 1
12 ESP32_ADDR = 0x88 # Connect to ESP32 Slave
13
14 btn_sos = Button(17)
15 btn_voice = Button(27)
16 time.sleep(15)
17
18 # Connect to Huskylens
19 hl = Huskylens(1)
20 hl.set_alg(ALGORITHM_OBJECT_RECOGNITION)
21 try:
22     bus = SMBus(I2C_BUS)
23     print("Done")
24 except:
25     print(f"I2C error {e}")
26
27 def speak(text): # Calling for the subprocess to enable TTS in Rasp Pi 5
28     print(f"Master sayings: {text}")
29     subprocess.Popen(["espeak-ng", "-v", "en-us+f2", "-s", "140", text])
30
31 # --- FUNCTIONS ---
32 def manual_sos():
33     speak("Emergency button pressed. Notifying family..")
34     try:
35         bus.write_byte(ESP32_ADDR, 1) #sending number 1 to esp32 for further processing
36         print("I2C Command Sent:SOS")
37     except Exception as e:
38         print(f"I2C ERROR : {e}")
39
40 def voice_report():
41     blocks = hl.get_blocks()
42     people = 0
43     for b in blocks:
44         if b.ID == 1: #This is to detect human
45             count = len(people)
46             msg = f"There are {count} people in front." if count > 0 else "Path is clear."
47             speak(msg)
48
49 speak("Welcome to Intelllicane")
50
51 try:
52     while True:
53         # 2. Check Buttons
54         if btn_sos.is_pressed():
55             manual_sos()
56             time.sleep(1) # Single debounce
57
58         if btn_voice.is_pressed():
59             voice_report()
60             time.sleep(1)
61
62     time.sleep(0.05)
63 except KeyboardInterrupt:
64     print("System Shutdown.")
```